Name (ID): ศุภณัฐ พลยมมา 2513160032

1. RSA Demonstration Lab

Please visit **https://www.cryptool.org/en/cto/rsa-step-by-step/** and go through all steps of the RSA Demonstration Lab offered by the tool.

Instructions:

- Try to play with the tool with two different p and q size.
- For example, in the first experiment you may set p=257 and q=593. In my case, the ϕ(n) is 151552, where I set my coprime (e) to 251, and the d is calculated as 84531. I have my private key of (152401, 84531) and the public key of (152401, 251).

Step 1: Set prime numbers

As a starting point for RSA, choose two prime numbers p and q:

$p =$	257
$q =$	593

For the algorithm to work, the two prime numbers must be different.

Name (ID): ศุภณัฐ พลยมมา 2513160032

Step 2: Calculate key values

The next steps will calculate the two keys.
Both consist of two numbers, one of which is equal.

- Private key: (n, d)
- Public key: (n, e)

> We call n the *RSA module*, e the *encryption exponent*, and d the *decryption exponent*.

---

n (Step 2.1)

To calculate n simply multiply p and q.

$n = p \cdot q$

$n = 257 \cdot 593$

$n = 152401$

> This means a length of 18 bit.

ϕ (Step 2.2)

To get the further values e and d the **Euler phi function** ϕ is used.
It calculates how many of the numbers less than or equal to n are **coprime** to n.
Coprime means that two numbers have no common divisor except 1.

Since p and q are different, you can use the simplified calculation formula:

$\phi(n) = (p - 1) \cdot (q - 1)$

$\phi(152401) = (257 - 1) \cdot (593 - 1)$

$\phi(152401) = 151552$

▶ Alternative calculation note

Name (ID): ศุภณัฐ พลยมมา 2513160032

e (Step 2.3)

The number e can now be chosen freely.
However, it must be **coprime** to $\phi(n)$.

$e =$	251

To check this, we can use the **gcd function**. It finds the "greatest common divisor".

The following condition must be true:

$$\gcd(e, \phi(n)) = 1$$

$$\gcd(251, 151552) = 1$$

This is true for the entered values.

▶ Reusability note

d (Step 2.4)

The number d is the **multiplicative inverse** to e.
This means that $d \cdot e$ results in 1 again if we compute $\mod \phi(n)$.

The **modulo function** $\mod$ calculates the remainder of a division.

Expressed in formulas, the following must apply:

$$(e \cdot d) \bmod \phi(n) = 1$$

d can be calculated with the **extended Euclidean algorithm**.
For the values entered, d can have the following value:

$$d = 84531$$

Name (ID): ศุภณัฐ พลยมมา 2513160032

The Resulting Key Pairs

⇒ Resulting keys

- Private key: $(n, d) = (152401, 84531)$
- Public key: $(n, e) = (152401, 251)$

Try to encrypt the message with following data 1, 10, and 100. Discuss the result.

Step 3: Encrypt

To encrypt a number m to ciphertext c the following formula is applied.
It uses the numbers of the public key:

$$c = m^e \bmod n$$

RSA encrypts only numbers. These must be greater-equal 0 and less than $n = 152401$.

Example:

$m =$	1

$$c = 1^{251} \bmod 152401$$

$$c = 1$$

Step 4: Decrypt

For decryption the inverse formula is applied.
It uses the numbers of the private key:

$$m' = c^d \bmod n$$

Example:

$c =$	1

$$m' = 1^{84531} \bmod 152401$$

$$m' = 1$$

Result Discussion:

Name (ID): ศุภณัฐ พลยมมา 2513160032

Example:

$m =$	1

$c = 1^{251} \bmod 152401$

$c = 1$

Letters are converted to numbers using an encoding system like **ASCII**.
For the entered values this would be: `m =` and `c =`

Step 4: Decrypt

For decryption the inverse formula is applied.
It uses the numbers of the private key:

$m' = c^d \bmod n$

Example:

$c =$	1

$m' = 1^{84531} \bmod 152401$

$m' = 1$

Example:

$m =$	10

$c = 10^{251} \bmod 152401$

$c = 81707$

Letters are converted to numbers using an encoding system like **ASCII**.
For the entered values this would be: `m =` and `c = 瓢`

Step 4: Decrypt

For decryption the inverse formula is applied.
It uses the numbers of the private key:

$m' = c^d \bmod n$

Example:

$c =$	10

$m' = 10^{84531} \bmod 152401$

$m' = 59124$

Name (ID): ศุภณัฐ พลยมมา 2513160032

$c = m^e \bmod n$

RSA encrypts only numbers. These must be greater-equal 0 and less than $n = 152401$.

Example:

$m =$	100

$c = 100^{251} \bmod 152401$

$c = 108044$

Letters are converted to numbers using an encoding system like **ASCII**.
For the entered values this would be: `m = d` and `c = =`

Step 4: Decrypt

For decryption the inverse formula is applied.
It uses the numbers of the private key:

$m' = c^d \bmod n$

Example:

$c =$	100

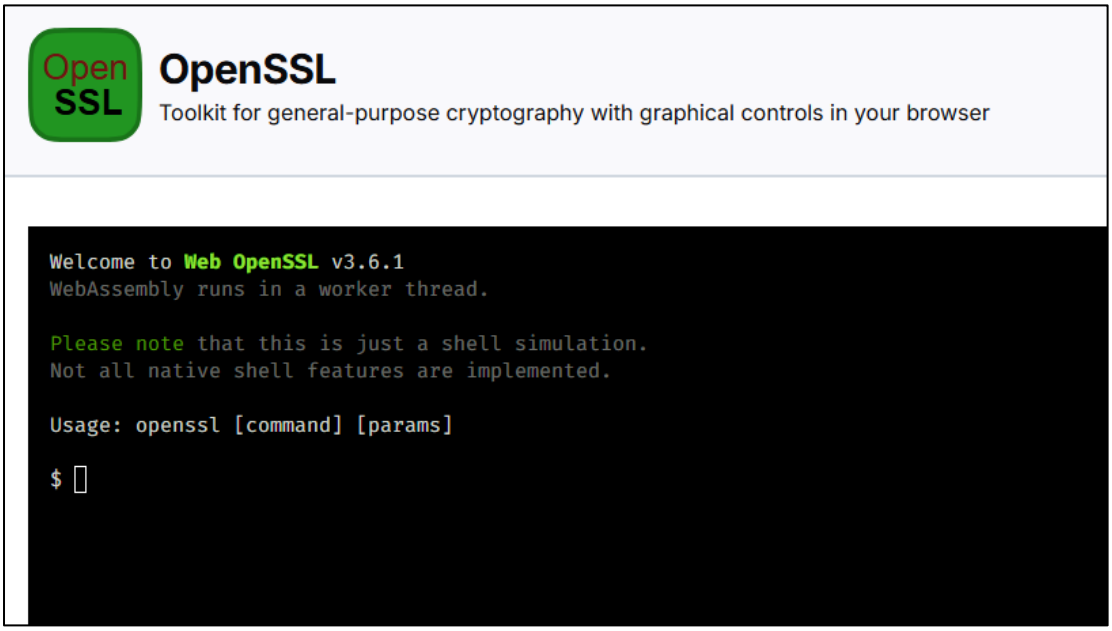
$m' = 100^{84531} \bmod 152401$

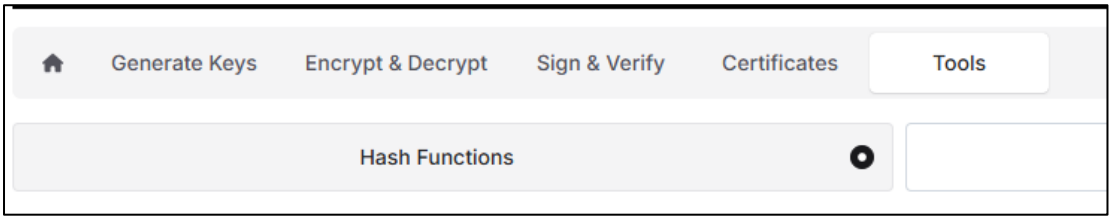
$m' = 25639$

Name (ID): ศุภณัฐ พลยมมา 2513160032

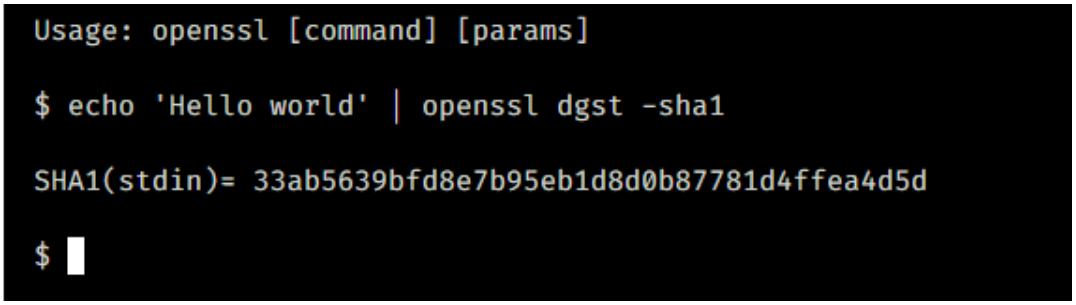
2. Play with OpenSSL

- Please visit https://www.cryptool.org/en/cto/openssl/ and follow the follow the lab instruction.
- Click on “Tools”
- At “Input” Key in “Hello world”





- Select hash function. In this example, we will go with **sha1**. Click “Run” and record the result.



Name (ID): ศุภณัฐ พลยมมา 2513160032

- Let's change the message to "**Hello World**"

```
$ echo 'Hello World' | openssl dgst -sha1

SHA1(stdin)= 648a6a6ffffdaa0badb23b8baf90b6168dd16b3a

$
```

Experiment:

Try to hash your own text or files using **different hash algorithms**, **capture the result screen and discuss following questions**.

Why hash algorithms give different output length?

Ans: **ความปลอดภัย:** ยิ่ง Output ยาว โอกาสที่ข้อมูลต่างกันจะสุ่มได้ค่าซ้ำกัน (Collision) ยิ่งน้อยลง ทำให้ระบบปลอดภัยจากการถูกเดาหรือปลอมแปลงข้อมูลได้ดีขึ้น

การออกแบบ: แต่ละ Algorithm มีสูตรคำนวณและโครงสร้างภายใน ที่กำหนดมาไม่เท่ากัน เพื่อให้เหมาะสมกับความแรงของ CPU ในยุคที่สร้างมันขึ้นมา

ความสมดุล: ความยาวที่ต่างกันช่วยให้เลือกใช้ได้ตามงาน เช่น งานเช็คไฟล์ทั่วไปใช้แบบสั้นเพื่อความรวดเร็ว ส่วนงานเก็บรหัสผ่านหรือ Blockchain จะใช้แบบยาวเพื่อความปลอดภัยสูงสุด

With the same input, will the different has function create similar output? Why?

Ans: **สูตรคณิตศาสตร์คนละชุด** แต่ละ Hash function (เช่น MD5, SHA-256) มีขั้นตอนการยำข้อมูล การเลื่อนบิต และค่าคงที่ทางคณิตศาสตร์ที่ถูกกำหนดมาต่างกัน ทำให้ผลลัพธ์สุดท้ายออกมาไม่เหมือนกัน

โครงสร้างภายในต่างกัน บางตัวเน้นการคำนวณที่รวดเร็ว (แบบสั้น) บางตัวเน้นการซ้อนทับหลายชั้นเพื่อความปลอดภัย (แบบยาว) ทำให้กระบวนการแปลงข้อมูลจาก Input ไปเป็น Output มีเส้นทางคนละสาย

Name (ID): ศุภณัฐ พลยมมา 2513160032

เป้าหมายคือความสุ่ม Hash ที่ดีถูกออกแบบมาให้ข้อมูลที่ออกมาดูสุ่ม ที่สุด ดังนั้นการที่ Algorithm สองตัวจะให้ผลลัพธ์คล้ายกันนั้นแทบจะเป็นไปไม่ได้